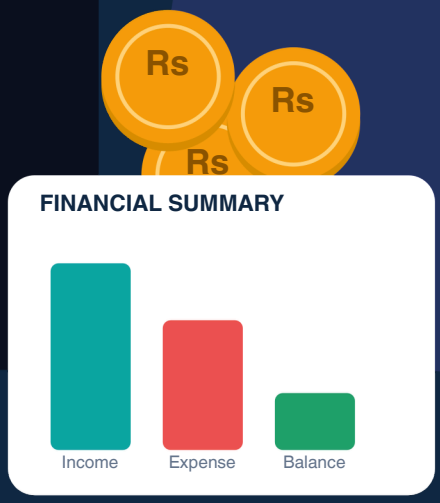
OOPS MINI-PROJECT REPORT

Smart Expense Tracker

Using C++ and Object-Oriented Programming

Console Application | C++17 | File Handling | OOP

```
================================
      SMART EXPENSE TRACKER
================================
1. Add Income
2. Add Expense
3. View All Transactions
4. View Financial Summary
5. Search by Category
6. Set Spending Budget
7. Exit
Enter your choice: 4
```



PROJECT DETAILS

Language	C++ (C++17, GCC / Clang)
Storage	Local text files (transactions.txt, budget.txt)
Editor	Visual Studio Code

SUBMITTED BY

Name : **Kavya** UID: **25BCD10086** Section: **BCD2'A'**

01 Project Overview

PROJECT TITLE	PROJECT TYPE	LANGUAGE	STORAGE
Smart Expense Tracker	Console-based application	C++ (C++17)	Local text file

Contents

01	Project Overview	2	09	System Design and Working	6
02	Abstract	2	10	Complete C++ Source Code	8
03	Introduction	3	11	Sample Outputs	13
04	Problem Statement	3	12	Testing and Results	14
05	Objectives	4	13	Conclusion	15
06	Scope of the Project	3	14	Future Scope	15
07	Software and Hardware Requirements	4	15	References	15
08	OOP Concepts Used	5			

02 Abstract

The **Smart Expense Tracker** is a C++ application designed to help users manage their personal finances by recording income and expenses. It allows users to categorize transactions, calculate total income and expenditure, check their available balance, and monitor spending against a budget.

The application uses **object-oriented programming** to organize financial data and operations into classes and member functions. It also uses **file handling** to save transactions so that the information can be accessed after the program is closed. Input validation and exception handling help prevent invalid entries.

The primary aim of this project is to demonstrate how C++ and OOP concepts can be applied to solve a common real-life problem. The system is suitable for students and individuals who want a simple way to monitor their spending without using a complex financial application.

Add Income | Add Expense | Categories | Budget Alerts | File Storage | Search | Validation

03 Introduction

Managing personal finances is an important part of everyday life. Students and working individuals often spend money on food, transport, shopping, education, entertainment, and other activities. When these transactions are not recorded properly, it becomes difficult to understand spending habits, control unnecessary expenses, and achieve savings goals.

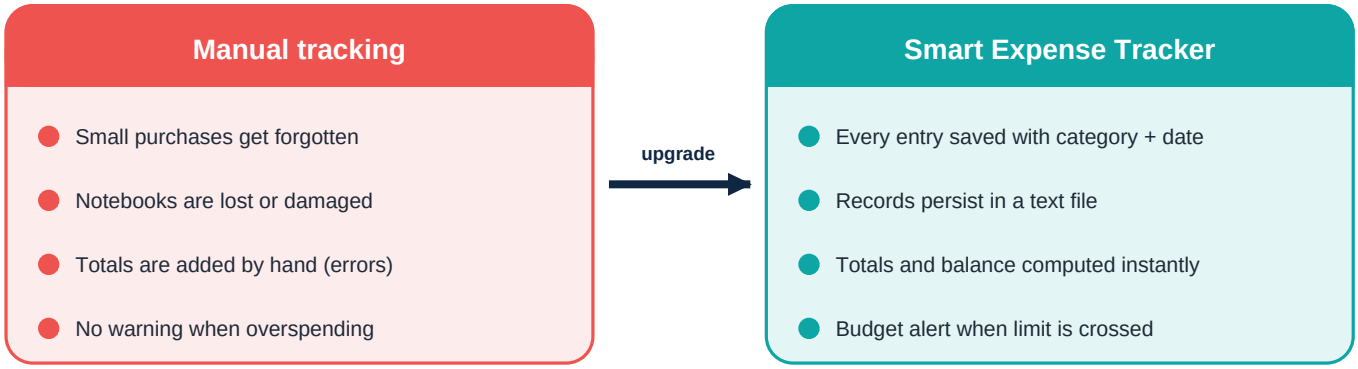
A Smart Expense Tracker provides a simple digital solution to this problem. Instead of maintaining handwritten records, users can enter their income and expenses into a computer program. The system calculates totals automatically and displays the remaining balance.

This project is developed using C++ and the principles of object-oriented programming. Classes and objects represent transactions and manage financial operations. File handling provides persistent storage, while vectors make it convenient to manage a collection of transactions during program execution. The project is designed for educational purposes and demonstrates how programming can be used to build a practical, organized, and reusable financial management system.

04 Problem Statement

Manual expense tracking can be time-consuming and prone to calculation errors. People may forget small transactions, lose written records, or fail to identify categories where they spend too much money. Basic note-taking methods also do not automatically calculate the remaining balance or compare spending against a budget.

Problem addressed: the lack of a simple, accessible system for recording and analyzing personal income and expenses.
Proposed solution: a menu-driven C++ application that records transactions, shows financial summaries, searches records, sets a budget, and saves data in a file.



06 Scope of the Project

Included in scope

- Adding income and expense transactions
- Storing date, category, amount and description
- Displaying all saved transactions
- Calculating income, expenses and balance
- Setting a budget and checking if expenses exceed it
- Searching transactions by category
- Saving and loading data through text files

Limitations

- Educational, single-user console application
- No bank-account connection or online payments
- No secure multi-user financial services
- Budget applies to all recorded expenses, not split by calendar month
- Dates are stored as text (not calendar-validated)
- Category search is case-sensitive

05 Objectives

The main objectives of the Smart Expense Tracker are:

1. Develop a user-friendly, menu-driven application using C++
2. Record income and expenses with amounts, categories and descriptions
3. Calculate total income, total expenses and available balance
4. Classify transactions into categories for easier analysis
5. Set a monthly budget and identify when expenses exceed it
6. Store transaction records using file handling
7. Apply OOP concepts: classes, objects, encapsulation, constructors
8. Validate user input and handle invalid entries
9. Encourage better budgeting and responsible spending

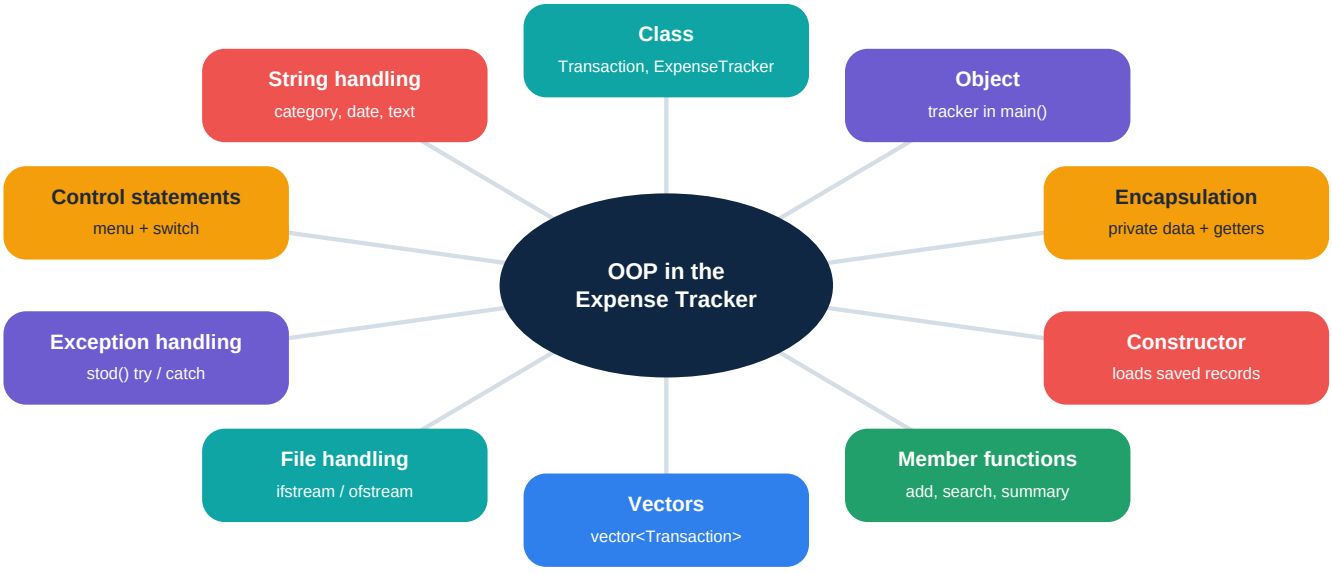
07 Software and Hardware Requirements

Component	Requirement
Programming language	C++
Compiler	GCC / Clang supporting C++17
Code editor	Visual Studio Code
Operating system	macOS, Windows, or Linux
Storage	Local text files
Libraries	Standard C++ library
Hardware	Basic computer with sufficient space to run VS Code and a C++ compiler

08 OOP Concepts Used

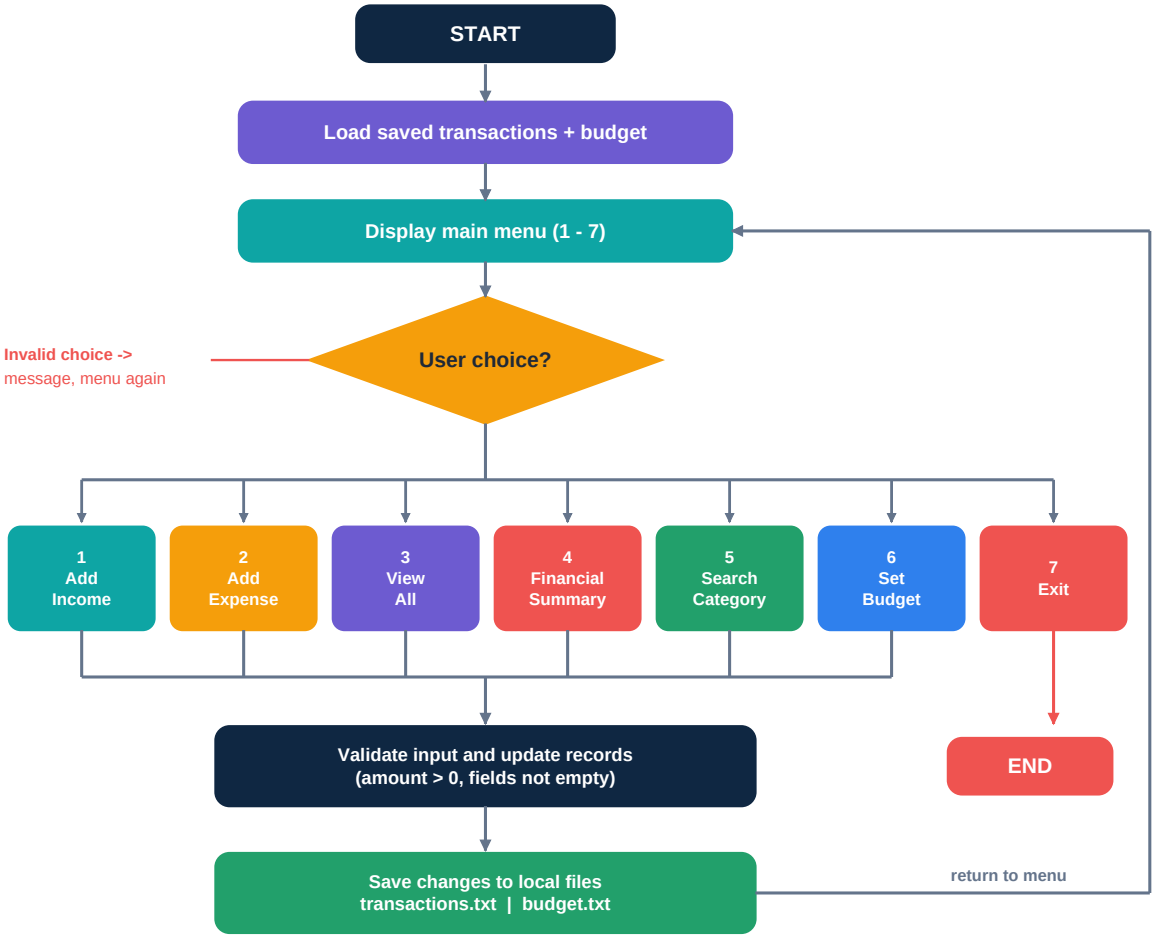
Concept	Application in this project
Class	Transaction and ExpenseTracker organize related data and operations.
Object	An ExpenseTracker object manages the application.
Encapsulation	Transaction details are kept private and accessed through member functions.
Constructor	Initializes the tracker and loads previously saved records.
Member functions	Add transactions, calculate totals, search records, and display summaries.
Vectors	Store transactions in memory while the program runs.
File handling	Save and retrieve transaction and budget data.
Exception handling	Handles invalid amounts and errors while converting stored numeric values.
Control statements	Menus and conditional statements control the program flow.
String handling	Stores descriptions, dates, categories, and transaction types.

Concept map

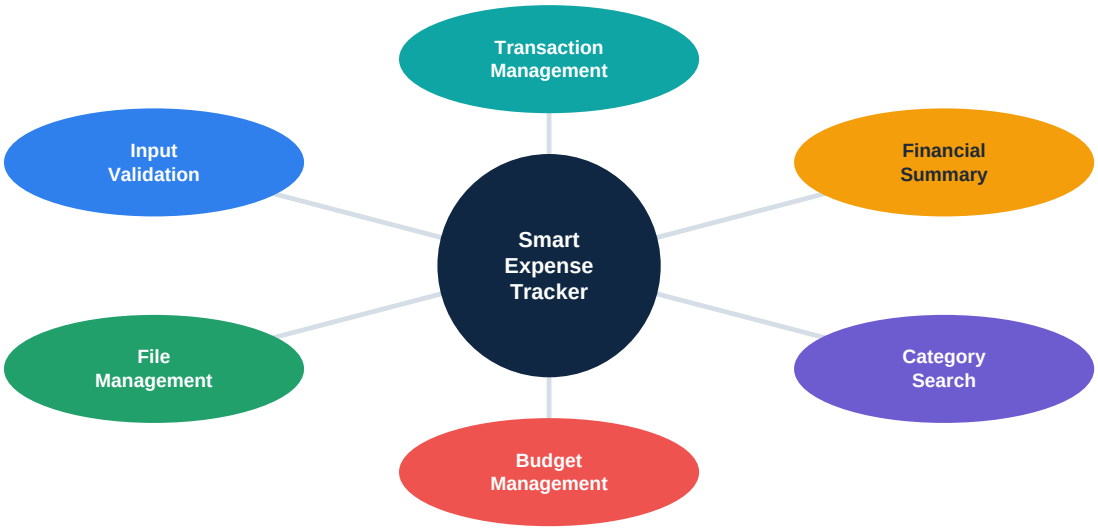


09 System Design and Working

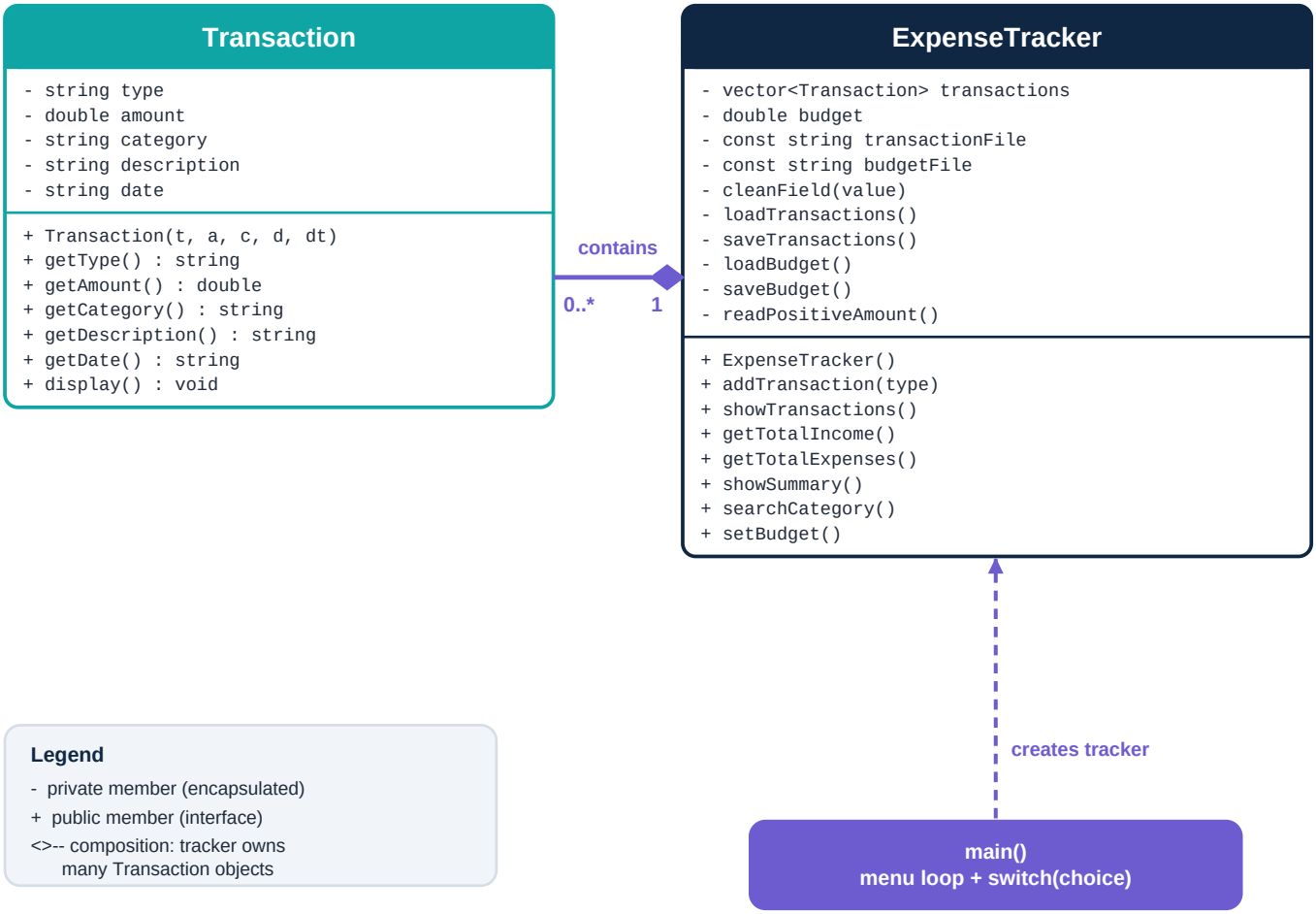
The program follows a simple sequence: it loads saved data, shows a menu, performs the selected operation, validates input, saves changes and returns to the menu until the user exits.



Main modules



Class diagram (UML)

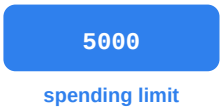


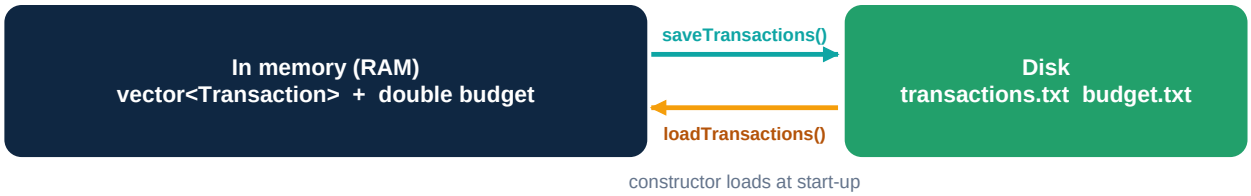
How data is stored on disk

transactions.txt (one record per line, fields separated by "|")



budget.txt





constructor loads at start-up

10 Complete C++ Source Code

Create a file named **main.cpp** and paste the following code into it. This version includes persistent storage, transaction management, category search, a spending budget and financial summaries. Compile with `g++ -std=c++17 main.cpp -o tracker`.

```cpp
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <string>
#include <iomanip>
#include <limits>
#include <stdexcept>

using namespace std;

class Transaction {
private:
    string type;
    double amount;
    string category;
    string description;
    string date;

public:
    Transaction(string t, double a, string c,
                string d, string dt)
        : type(t), amount(a), category(c),
          description(d), date(dt) {}

    string getType() const { return type; }
    double getAmount() const { return amount; }
    string getCategory() const { return category; }
    string getDescription() const { return description; }
    string getDate() const { return date; }

    void display() const {
        cout << left << setw(12) << type
             << setw(12) << fixed << setprecision(2)
             << amount
             << setw(16) << category
             << setw(16) << date
             << description << '\n';
    }
};

class ExpenseTracker {
private:
    vector<Transaction> transactions;
    double budget;
    const string transactionFile = "transactions.txt";
    const string budgetFile = "budget.txt";

    string cleanField(string value) {
        for (char &ch : value) {
            if (ch == '|')
                ch = '/';
        }
        return value;
    }

    void loadTransactions() {
        ifstream file(transactionFile);
        string line;

        while (getline(file, line)) {
            stringstream ss(line);
            string type, amountText, category;
            string description, date;

            if (getline(ss, type, '|') &&
                getline(ss, amountText, '|') &&
                getline(ss, category, '|') &&
                getline(ss, description, '|') &&
                getline(ss, date)) {
                try {
                    double amount = stod(amountText);
                    if (amount > 0) {
                        transactions.emplace_back(
                            type, amount, category,
                            description, date);
                    }
                } catch (const exception &) {
                    // Skip invalid or corrupted records.
                }
            }
        }
    }

    void saveTransactions() const {
```

```
        ofstream file(transactionFile);

        if (!file) {
            cout << "Error saving transactions!\n";
            return;
        }

        for (const auto &t : transactions) {
            file << t.getType() << '|'
                 << t.getAmount() << '|'
                 << t.getCategory() << '|'
                 << t.getDescription() << '|'
                 << t.getDate() << '\n';
        }
    }

    void loadBudget() {
        ifstream file(budgetFile);
        if (!(file >> budget) || budget < 0)
            budget = 0;
    }

    void saveBudget() const {
        ofstream file(budgetFile);
        if (file)
            file << budget;
        else
            cout << "Error saving budget!\n";
    }

    double readPositiveAmount() {
        double amount;

        while (true) {
            cout << "Enter amount (greater than 0): ";

            if (cin >> amount && amount > 0) {
                cin.ignore(numeric_limits<streamsize>::max(),
                           '\n');
                return amount;
            }

            cout << "Invalid amount. Try again.\n";
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(),
                       '\n');
        }
    }

public:
    ExpenseTracker() : budget(0) {
        loadTransactions();
        loadBudget();
    }

    void addTransaction(const string &type) {
        double amount = readPositiveAmount();
        string category, description, date;

        cout << "Enter category (Food/Travel/Shopping/"
                "Education/Other): ";
        getline(cin, category);

        cout << "Enter description: ";
        getline(cin, description);

        cout << "Enter date (DD-MM-YYYY): ";
        getline(cin, date);

        if (category.empty() || description.empty()
            || date.empty()) {
            cout << "Fields cannot be empty. "
                    "Transaction cancelled.\n";
            return;
        }

        transactions.emplace_back(
            type, amount, cleanField(category),
            cleanField(description), cleanField(date));

        saveTransactions();

        cout << type << " recorded successfully!\n";

        if (budget > 0 && getTotalExpenses() > budget) {
            cout << "Warning: Your spending budget "
                    "has been exceeded!\n";
        }
    }

    void showTransactions() const {
        if (transactions.empty()) {
```

```
            cout << "No transactions recorded yet.\n";
            return;
        }

        cout << "\n--- TRANSACTION HISTORY ---\n";
        cout << left << setw(12) << "Type"
             << setw(12) << "Amount"
             << setw(16) << "Category"
             << setw(16) << "Date"
             << "Description\n";

        for (const auto &t : transactions)
            t.display();
    }

    double getTotalIncome() const {
        double total = 0;

        for (const auto &t : transactions) {
            if (t.getType() == "Income")
                total += t.getAmount();
        }

        return total;
    }

    double getTotalExpenses() const {
        double total = 0;

        for (const auto &t : transactions) {
            if (t.getType() == "Expense")
                total += t.getAmount();
        }

        return total;
    }

    void showSummary() const {
        double income = getTotalIncome();
        double expenses = getTotalExpenses();

        cout << fixed << setprecision(2);
        cout << "\n--- FINANCIAL SUMMARY ---\n";
        cout << "Total Income:   Rs. " << income << '\n';
        cout << "Total Expenses: Rs. " << expenses << '\n';
        cout << "Balance:        Rs. "
             << income - expenses << '\n';

        if (budget > 0) {
            cout << "Spending Budget: Rs. " << budget << '\n';
            cout << "Budget Remaining: Rs. "
                 << budget - expenses << '\n';

            if (expenses > budget)
                cout << "Status: Budget exceeded!\n";
            else
                cout << "Status: Within budget.\n";
        }
    }

    void searchCategory() const {
        string category;
        bool found = false;

        cout << "Enter category to search: ";
        getline(cin, category);

        for (const auto &t : transactions) {
            if (t.getCategory() == category) {
                if (!found) {
                    cout << "\nMatching transactions:\n";
                    found = true;
                }
                t.display();
            }
        }

        if (!found)
            cout << "No transactions found in this category.\n";
    }

    void setBudget() {
        cout << "Set spending budget.\n";
        budget = readPositiveAmount();
        saveBudget();

        cout << "Budget saved successfully!\n";
        if (getTotalExpenses() > budget)
            cout << "Warning: Existing expenses exceed "
                    "your new budget!\n";
    }
};
```

```cpp

int main() {
    ExpenseTracker tracker;
    int choice;

    do {
        cout << "\n================================\n";
        cout << "      SMART EXPENSE TRACKER\n";
        cout << "================================\n";
        cout << "1. Add Income\n";
        cout << "2. Add Expense\n";
        cout << "3. View All Transactions\n";
        cout << "4. View Financial Summary\n";
        cout << "5. Search by Category\n";
        cout << "6. Set Spending Budget\n";
        cout << "7. Exit\n";
        cout << "Enter your choice: ";

        if (!(cin >> choice)) {
            cout << "Invalid input. Enter a number.\n";
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(),
                       '\n');
            continue;
        }

        cin.ignore(numeric_limits<streamsize>::max(), '\n');

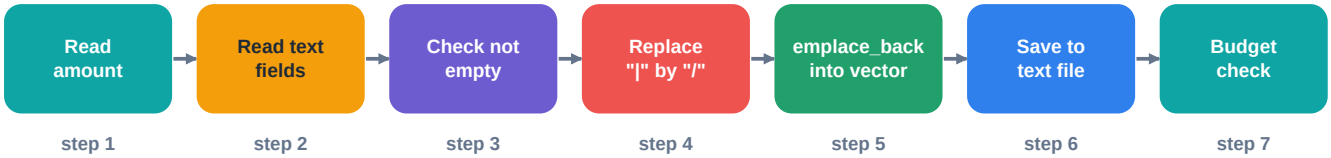
        switch (choice) {
            case 1:
                tracker.addTransaction("Income");
                break;
            case 2:
                tracker.addTransaction("Expense");
                break;
            case 3:
                tracker.showTransactions();
                break;
            case 4:
                tracker.showSummary();
                break;
            case 5:
                tracker.searchCategory();
                break;
            case 6:
                tracker.setBudget();
                break;
            case 7:
                cout << "Thank you for using "
                        "Smart Expense Tracker!\n";
                break;
            default:
                cout << "Invalid choice. Try again.\n";
        }

    } while (choice != 7);

    return 0;
}
```

Explanation of the code

What addTransaction() does, step by step



Code element	Role
Transaction	Represents one income or expense record and keeps its data private.
ExpenseTracker	Manages the transaction collection, calculations, searching, budgeting and file operations.
vector<Transaction>	Stores records in memory while the application runs.
transactions.txt / budget.txt	Keep the history and the spending budget so they survive a restart.
getTotalIncome() / getTotalExpenses()	Calculate totals by iterating through the transaction vector.
showSummary()	Calculates balance as total income minus total expenses.
searchCategory()	Searches for an exact category match.
readPositiveAmount()	Rejects non-numeric or non-positive amounts and asks again.
Constructor	Loads existing data when the application starts.

Important implementation note: the program stores dates as text and does not validate calendar dates or divide the budget into monthly periods. Category searches are case-sensitive, so *Food* and *food* are treated as different categories.

11 Sample Outputs

The outputs below were captured by compiling the code above (`g++ -std=c++17`) and driving it with the sample data in this report. Amber text is what the user types.

Output 1 - Main menu

```
tracker
==================================
      SMART EXPENSE TRACKER
==================================
1. Add Income
2. Add Expense
3. View All Transactions
4. View Financial Summary
5. Search by Category
6. Set Spending Budget
7. Exit
Enter your choice:
```

Output 2 - Adding income

```
tracker
Enter your choice: 1
Enter amount (greater than 0): 10000
Enter category (Food/Travel/Shopping/Education/Other): Other
Enter description: Monthly allowance
Enter date (DD-MM-YYYY): 01-10-2026
Income recorded successfully!
```

Output 3 - Adding an expense

```
tracker
Enter your choice: 2
Enter amount (greater than 0): 1500
Enter category (Food/Travel/Shopping/Education/Other): Food
Enter description: Monthly groceries
Enter date (DD-MM-YYYY): 02-10-2026
Expense recorded successfully!
```

Output 4 - Input validation (negative / text amount)

```
tracker
Enter your choice: 2
Enter amount (greater than 0): -200
Invalid amount. Try again.
Enter amount (greater than 0): abc
Invalid amount. Try again.
Enter amount (greater than 0): 1200
Enter category (Food/Travel/Shopping/Education/Other): Shopping
Enter description: Headphones
Enter date (DD-MM-YYYY): 12-10-2026
Expense recorded successfully!
```

Output 5 - Transaction history

```
tracker
Enter your choice: 3

--- TRANSACTION HISTORY ---
Type        Amount      Category    Date          Description
Income      10000.00    Other       01-10-2026    Monthly allowance
Expense     1500.00     Food        02-10-2026    Monthly groceries
Expense     800.00      Travel      05-10-2026    Metro pass
Expense     450.00      Food        09-10-2026    Dinner with friends
Expense     1200.00     Shopping    12-10-2026    Headphones
```

Invalid menu choice

```
tracker
Enter your choice: 9
Invalid choice. Try again.
```

Output 6 - Financial summary (within budget)

```
tracker
Enter your choice: 4

--- FINANCIAL SUMMARY ---
Total Income:   Rs. 10000.00
Total Expenses: Rs. 3950.00
Balance:        Rs. 6050.00
Spending Budget: Rs. 5000.00
Budget Remaining: Rs. 1050.00
Status: Within budget.
```

Output 7 - Search by category: Food

```
tracker
Enter your choice: 5
Enter category to search: Food

Matching transactions:
Expense     1500.00     Food          02-10-2026     Monthly groceries
Expense     450.00      Food          09-10-2026     Dinner with friends
```

Output 8 - Budget exceeded warning

```
tracker
Enter your choice: 2
Enter amount (greater than 0): 3000
Enter category (Food/Travel/Shopping/Education/Other): Education
Enter description: Course fee
Enter date (DD-MM-YYYY): 15-10-2026
Expense recorded successfully!
Warning: Your spending budget has been exceeded!

Enter your choice: 4

--- FINANCIAL SUMMARY ---
Total Income:   Rs. 10000.00
Total Expenses: Rs. 6950.00
Balance:        Rs. 3050.00
Spending Budget: Rs. 5000.00
Budget Remaining: Rs. -1950.00
Status: Budget exceeded!
```

Output 9 - Data persisted on disk (survives restart)

```
bash
$ cat transactions.txt
Income|10000|Other|Monthly allowance|01-10-2026
Expense|1500|Food|Monthly groceries|02-10-2026
Expense|800|Travel|Metro pass|05-10-2026
Expense|450|Food|Dinner with friends|09-10-2026
Expense|1200|Shopping|Headphones|12-10-2026
Expense|3000|Education|Course fee|15-10-2026

$ cat budget.txt
5000
```

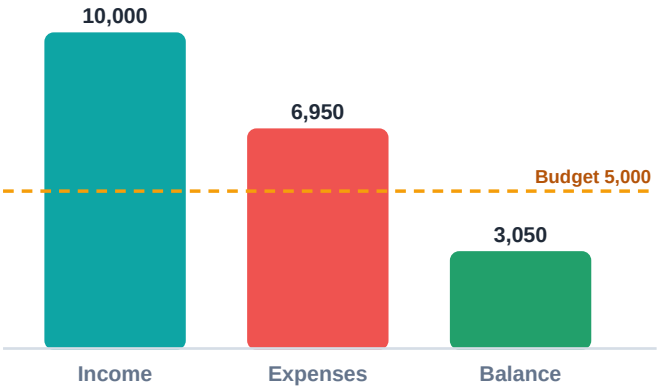
12 Testing and Results

Testing checks whether the application performs its intended operations correctly. All nine cases were executed on the compiled program.

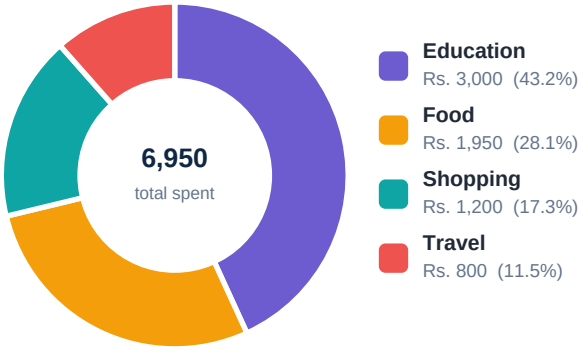
#	Test case	Input or action	Expected result	Result
1	**Add income**	Enter Rs. 10,000	Income record saved	PASS
2	**Add expense**	Enter Rs. 1,500	Expense record saved	PASS
3	**Calculate balance**	Income Rs. 10,000; expense Rs. 1,500	Balance Rs. 8,500 (income - expenses)	PASS
4	**Set budget**	Enter Rs. 5,000	Budget saved	PASS
5	**Exceed budget**	Record expenses exceeding Rs. 5,000	Warning displayed	PASS
6	**Search category**	Search for Food	Matching food transactions displayed	PASS
7	**Invalid amount**	Enter a negative amount	Program requests valid input	PASS
8	**Restart program**	Exit and reopen	Previously saved records are loaded	PASS
9	**Invalid menu choice**	Enter an unsupported option	Error message displayed	PASS

Visual results from the sample session

Income vs Expenses vs Balance (Rs.)



Spending by category



Budget usage: Rs. 5,000 limit



Result: the Smart Expense Tracker demonstrates that a console-based application can record financial transactions, calculate totals, manage a spending limit, search by category, and preserve information using file handling. The project applies core C++ and OOP concepts to a practical problem and provides a foundation for more advanced financial management applications.

13 Conclusion

The Smart Expense Tracker was developed to simplify the process of recording and monitoring personal finances. The system provides a menu-driven interface for adding income and expenses, viewing transaction records, calculating balances, searching categories, and setting a spending budget.

The project demonstrates the use of classes, objects, encapsulation, constructors, vectors, file streams, string handling, and input validation in C++. Persistent storage ensures that the recorded information is available across program sessions.

Overall, this mini project provides practical experience in designing and implementing an object-oriented program. Future improvements could include monthly reports, expense charts, transaction editing and deletion, password protection, and a graphical user interface.

14 Future Scope



Feature	Description
Monthly reports	Separate income and expenses by month.
Visual analytics	Display spending through bar charts or pie charts.
Edit and delete	Allow users to correct or remove records.
Savings goals	Track progress toward a target amount.
Advanced search	Filter by date, amount, and multiple categories.
Graphical interface	Develop a GUI for easier navigation.
Security	Add authentication and stronger protection for stored financial data.

15 References

1 Bjarne Stroustrup, *The C++ Programming Language*, Addison-Wesley.

2 cppreference.com - C++ language and standard library.

3 LearnCpp - C++ tutorials.

4 Visual Studio Code - C++ programming documentation.

5 GitHub Docs - Getting started with Git.